Razvoj web aplikacija - Predavanje 5
API dizajn, validacija i CRUD domene

mag.ing. Josip Torić (SIT UNIZD)

23.03.2026.

Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

1. Modeliranje domene: Lifter, Competition, Registration
2. CRUD kroz slojeve: router $\rightarrow$ service $\rightarrow$ repository $\rightarrow$ model
3. Pydantic schemed: Create, Update, Response varijante
4. Validacija i error odgovori (422 i custom errori)
5. Paginacija i filtriranje
6. Partial update (PATCH vs PUT)

Na kraju predavanja imamo kompletne CRUD endpointe za sve entitete domene.

Ishodi učenja

Nakon ovog predavanja student može:

- modelirati složenije relacije (Lifter, Competition, Registration)
- dizajnirati REST API s konzistentnim URL-ovima i status codeovima
- razlikovati Create, Update i Response schema
- implementirati validaciju u Pydantic schemi
- objasniti PATCH vs PUT i kad koristiti koji
- implementirati paginaciju s limit/offset

Kontekst: što imamo, što fali

Imamo (P1-P4)

- Club, User modeli
- Auth (JWT login/refresh)
- Role + ownership provjere
- Error handling framework

Danas dodajemo

- Lifter, Competition, Registration
- CRUD za sve entitete
- Schemed (DTO)
- Validacija i paginacija

Novi entiteti: ER dijagram (tekst)

- **Club** 1:N **Lifter** - klub ima više natjecatelja
- **Club** 1:1 **User** - klub ima jednog korisnika za prijavu
- **Competition** 1:N **Registration** - natjecanje ima više prijava
- **Lifter** 1:N **Registration** - natjecatelj može biti prijavljen na više natjecanja

Registration je **veza** između Liftera i Competitiona - s dodatnim podacima (kategorija, datum prijave).

Model: Lifter

```
class Lifter(Base):
    __tablename__ = "lifters"

    id: Mapped[int] = mapped_column(primary_key=True)
    first_name: Mapped[str] = mapped_column(String(80))
    last_name: Mapped[str] = mapped_column(String(80))
    birth_date: Mapped[date] = mapped_column(Date)
    gender: Mapped[str] = mapped_column(String(1)) # M/F

    club_id: Mapped[int] = mapped_column(
        ForeignKey("clubs.id"), nullable=False
    )
    club: Mapped["Club"] = relationship(back_populates="lifters")
    registrations: Mapped[list["Registration"]] = relationship(
        back_populates="lifter"
    )
```

Model: Competition

```
class Competition(Base):
    __tablename__ = "competitions"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(200), nullable=False)
    date: Mapped[date] = mapped_column(Date, nullable=False)
    location: Mapped[str] = mapped_column(String(200))

    prelim_deadline: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), nullable=False
    )
    final_deadline: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), nullable=False
    )

    registrations: Mapped[list["Registration"]] = relationship(
        back_populates="competition"
    )
```

Dva roka: `prelim_deadline` i `final_deadline`. Pravila rokova dolaze u Predavanju 6.

Model: Registration

```
class Registration(Base):
    __tablename__ = "registrations"
    __table_args__ = (
        UniqueConstraint("lifter_id", "competition_id",
                         name="uq_lifter_competition"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    lifter_id: Mapped[int] = mapped_column(
        ForeignKey("lifters.id"), nullable=False
    )
    competition_id: Mapped[int] = mapped_column(
        ForeignKey("competitions.id"), nullable=False
    )
    category: Mapped[str] = mapped_column(String(20))
    status: Mapped[str] = mapped_column(
        String(20), default="active"
    ) # active / withdrawn
    registered_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
```

`UniqueConstraint` sprječava duplu prijavu istog liftera na isto natjecanje.

Unique constraint na Registration: zašto?

Bez `UniqueConstraint("lifter_id", "competition_id")`:

- dva konkurentna requesta mogu prijaviti istog liftera dva puta
- `if` u kodu ne štiti od race conditiona
- baza mora biti autoritet za integritet

Kad se constraint prekrši, baza baca `IntegrityError`. Service ga hvata i vraća 409 Conflict.

Registracija novih modela

Svaki novi model mora biti importiran u `models/__init__.py`:

Datoteka	Zašto
`models/__init__.py`	da Alembic vidi metadata
`main.py`	novi router se registrira s `include_router`

Ako zaboraviš `models/__init__.py`, `alembic revision --autogenerate` neće detektirati novu tablicu.

REST API dizajn: konzistentnost

Metoda	URL	Akcija	Status
GET	/clubs/{id}/lifters	lista	200
POST	/clubs/{id}/lifters	kreiraj	201
GET	/clubs/{id}/lifters/{id}	dohvati	200
PATCH	/clubs/{id}/lifters/{id}	djelomično ažuriraj	200
DELETE	/clubs/{id}/lifters/{id}	obriši	204

Nested URL-ovi: resurs živi **pod** roditeljskim resursom. Ownership je implicitan u URL-u.

Schema dizajn: zašto tri varijante?

Schema	Svrha
`LifterCreate`	Input za `POST`. Sadrži samo polja koja klijent šalje (nema `id`, `registered_at`).
`LifterUpdate`	Input za `PATCH`. Sva polja su opcionalna (`Optional`).
`LifterResponse`	Output. Sadrži sve relevantne podatke uključujući `id`.

Nikad ne koristimo isti DTO za input i output. Input validira, output prezentira.

Schema: LifterCreate i LifterResponse

```
from pydantic import BaseModel, Field, field_validator
from datetime import date
from typing import Literal

class LifterCreate(BaseModel):
    first_name: str = Field(min_length=1, max_length=80)
    last_name: str = Field(min_length=1, max_length=80)
    birth_date: date
    gender: Literal["M", "F"]

    @field_validator("birth_date")
    @classmethod
    def must_be_past(cls, v):
        if v >= date.today():
            raise ValueError("Birth date must be in the past")
        return v

class LifterResponse(BaseModel):
    id: int
    first_name: str
    last_name: str
    birth_date: date
    gender: str
    club_id: int

    model_config = {"from_attributes": True}
```

Schema: LifterUpdate (partial update)

```
class LifterUpdate(BaseModel):
    first_name: str | None = Field(
        None, min_length=1, max_length=80
    )
    last_name: str | None = Field(
        None, min_length=1, max_length=80
    )
    birth_date: date | None = None
    gender: str | None = Field(None, pattern="^[MF]$")
```

Sva polja su None po defaultu. U serviceu ažuriramo samo ona koja su poslana:

```
for field, value in body.model_dump(exclude_unset=True).items():
    setattr(lifter, field, value)
```

PATCH vs PUT

	PUT	**PATCH**
Semantika	zamijeni cijeli resurs	ažuriraj samo poslana polja
Validacija	sva polja obavezna	samo poslana polja
Kad koristiti	rijetko u SPA	gotovo uvijek

U praksi, SPA gotovo uvijek šalje parcijalne podatke. PATCH je prirodniji izbor.

Repository: CRUD operacije

```
async def get_by_id(db: AsyncSession, lifter_id: int) -> Lifter | None:
    result = await db.execute(
        select(Lifter).where(Lifter.id == lifter_id)
    )
    return result.scalar_one_or_none()

async def create(db: AsyncSession, lifter: Lifter) -> Lifter:
    db.add(lifter)
    await db.flush()
    return lifter

async def delete(db: AsyncSession, lifter: Lifter) -> None:
    await db.delete(lifter)

async def get_by_club(
    db: AsyncSession, club_id: int
) -> list[Lifter]:
    result = await db.execute(
        select(Lifter).where(Lifter.club_id == club_id)
    )
    return list(result.scalars().all())
```

flush vs commit

- `flush()` - šalje SQL u bazu, ali **ne committa** transakciju
  - dodjeljuje `id` objektu
  - aktivira constraint-e (IntegrityError se pojavi ovdje)
- `commit()` - potvrđuje transakciju
  - radi `get_db()` dependency automatski na kraju requesta

U repositoryju radimo `flush()`, commit ostavljamo dependency-ju.

Service: CRUD s ownership provjerom

```
async def create_lifter(
    db: AsyncSession, club_id: int,
    body: LifterCreate, current_user: User,
) -> Lifter:
    if current_user.role == "club":
        if current_user.club_id != club_id:
            raise AppError("forbidden", "Pristup odbijen", 403)

    lifter = Lifter(
        first_name=body.first_name,
        last_name=body.last_name,
        birth_date=body.birth_date,
        gender=body.gender,
        club_id=club_id,
    )
    return await lifter_repo.create(db, lifter)
```

Router: kompletni CRUD

```
@router.post("/", status_code=201, response_model=LifterResponse)
async def create_lifter(
    club_id: int,
    body: LifterCreate,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
):
    return await lifter_service.create_lifter(db, club_id, body, user)

@router.get("/{lifter_id}", response_model=LifterResponse)
async def get_lifter(
    club_id: int,
    lifter_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    return await lifter_service.get_lifter(db, club_id, lifter_id, user)

@router.delete("/{lifter_id}", status_code=204)
async def delete_lifter(
    club_id: int,
    lifter_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_role("admin", "club")),
):
    await lifter_service.delete_lifter(db, club_id, lifter_id, user)
```

Validacija: što Pydantic radi automatski

- Tipovi: `str`, `int`, `date` - ako ne pase, 422
- Constraints: `min_length`, `max_length`, `pattern`
- Required vs optional polja
- Nested modeli

Pydantic 422 response uključuje `detail` s opisom svakog polja koje ne prolazi validaciju. Frontend može prikazati greške po polju.

Pydantic: custom validator

```
from pydantic import field_validator

class CompetitionCreate(BaseModel):
    name: str = Field(min_length=1, max_length=200)
    date: date
    prelim_deadline: datetime
    final_deadline: datetime

    @field_validator("final_deadline")
    @classmethod
    def final_after_prelim(cls, v, info):
        prelim = info.data.get("prelim_deadline")
        if prelim and v <= prelim:
            raise ValueError(
                "final_deadline mora biti nakon prelim_deadline"
            )
        return v
```

Validator se poziva pri kreiranju schemed. Ako ne prođe - 422 s porukom.

Schema: RegistrationCreate s kategorijama

```
MALE_CATEGORIES = ["59", "66", "74", "83", "93", "105", "120", "120+"]
FEMALE_CATEGORIES = ["47", "52", "57", "63", "69", "76", "84", "84+"]

class RegistrationCreate(BaseModel):
    lifter_id: int
    category: str

    @field_validator("category")
    @classmethod
    def valid_category(cls, v):
        all_cats = MALE_CATEGORIES + FEMALE_CATEGORIES
        if v not in all_cats:
            raise ValueError(f"Invalid category: {v}")
        return v
```

Kategorije prate IPF standarde. Cross-validaciju (spol natjecatelja vs kategorija) radimo u **service sloju**, ne u schemi.

Hvatanje IntegrityError

Kad baza odbije operaciju (unique, foreign key), SQLAlchemy baca `IntegrityError`.

Constraint	Status	Poruka
Unique	409	„Natjecatelj je vec prijavljen na ovo natjecanje”
Foreign key	400	„Klub ne postoji”
NOT NULL	422	obično uhvati Pydantic prije baze

Service hvata `IntegrityError` i pretvara u `AppError` s razumljivom porukom.

Paginacija: limit + offset

```
@router.get("/", response_model=list[LifterResponse])
async def list_lifters(
    club_id: int,
    db: AsyncSession = Depends(get_db),
    user: User = Depends(get_current_user),
    limit: int = Query(default=20, le=100),
    offset: int = Query(default=0, ge=0),
):
    return await lifter_service.list_lifters(
        db, club_id, user, limit=limit, offset=offset
    )
```

```
# u repositoryju
stmt = select(Lifter).limit(limit).offset(offset)
```

Paginacija: limit/offset vs cursor

	Limit/offset	**Cursor**
Implementacija	jednostavna	složenija
Kad se podaci mijenjaju	može preskočiti/duplikat	stabilan
Kad koristiti	mali skupovi, admin UI	feed, veliki skupovi

Za naš projekt limit/offset je sasvim dovoljan.

Filtriranje: query parametri

Primjer: `GET /clubs/3/lifters?gender=M`

- `club_id` dolazi iz URL patha (nested URL)
- Dodatni filteri (`gender`) su query parametri - opcionalni
- Repository gradi `WHERE` klauzulu dinamički
- Ne treba posebni search engine - SQLAlchemy je dovoljan za ovo

Filtriranje u repositoryju

```
async def list_lifters(
    db: AsyncSession,
    club_id: int,
    gender: str | None = None,
    limit: int = 20,
    offset: int = 0,
) -> list[Lifter]:
    stmt = select(Lifter).where(Lifter.club_id == club_id)
    if gender is not None:
        stmt = stmt.where(Lifter.gender == gender)
    stmt = stmt.limit(limit).offset(offset)
    result = await db.execute(stmt)
    return list(result.scalars().all())
```

Commitovi za Predavanje 5

- `feat(api): add Lifter, Competition, Registration models + migration`
- `feat(api): add schemas for all entities (create/update/response) with validators`
- `feat(api): implement lifter CRUD (nested /clubs/{id}/lifters) with pagination`
- `feat(api): implement competition CRUD (admin-write, all-read)`
- `feat(api): implement registration CRUD (nested /competitions/{id}/registrations)`
- `test(api): add CRUD, validation, constraint, duplicate and ownership tests`

Na satu: prolazimo diffeve. Testiramo u Swaggeru: CRUD, validacija, paginacija, nested URL-ovi.

Mini checklista: što mora raditi do kraja sata

1. Novi modeli u bazi (`alembic upgrade head`)
2. CRUD endpointi za Lifter, Competition, Registration
3. Pydantic validacija vraća 422 za krivi input
4. Unique constraint vraća 409 za duplu registraciju
5. Paginacija radi na listama

Što ide sljedeće (Predavanje 6)

- Faze natjecanja: OPEN, PRELIM_PASSED, CLOSED
- Tko smije što u kojoj fazi (nova prijava, odjava, promjena kategorije)
- Soft status: odjava je `status = "withdrawn"`, ne DELETE
- `freezegun` za testiranje vremenski ovisne logike
- **Pred obranu 1** — sve mora raditi

Pitanja?